

ASP.NET Core Extension Method vs Normal Static Method

1. Normal Static Method

ServiceCollectionExtensions.cs

```
using SwiggyAPI.API.Implementations;
using SwiggyAPI.API.Interfaces;

namespace SwiggyAPI.API.Services;

public static class ServiceCollectionExtensions
{
    public static IServiceCollection AddApplicationServices(IServiceCollection services)
    {
        // Whenever someone asks for IRestaurantService,
        // give them RestaurantService
        services.AddScoped<IRestaurantService, RestaurantService>();

        return services;
    }
}
```

Program.cs

```
using SwiggyAPI.API.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

// Calling normal static method
ServiceCollectionExtensions.AddApplicationServices(builder.Services);

var app = builder.Build();

app.UseHttpsRedirection();

app.UseAuthorization();
```

```
app.MapControllers();
```

```
app.Run();
```

How This Works

This is a normal static method.

Because method parameter is:

```
IServiceCollection services
```

There is NO `this` keyword.

So method belongs to:

```
ServiceCollectionExtensions
```

Hence we call:

```
ServiceCollectionExtensions.AddApplicationServices(builder.Services);
```

Flow

```
Program.cs
```

```
↓
```

```
Calls static method
```

```
↓
```

```
AddApplicationServices(builder.Services)
```

```
↓
```

```
Registers:
```

```
IRestaurantService → RestaurantService
```

```
↓
```

```
DI container stores mapping
```

2. Extension Method

ServiceCollectionExtensions.cs

```
using SwiggyAPI.API.Implementations;
using SwiggyAPI.API.Interfaces;

namespace SwiggyAPI.API.Services;

public static class ServiceCollectionExtensions
{
    public static IServiceCollection AddApplicationServices(
        this IServiceCollection services)
    {
        services.AddScoped<IRestaurantService, RestaurantService>();

        return services;
    }
}
```

Program.cs

```
using SwiggyAPI.API.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

// Calling extension method
builder.Services.AddApplicationServices();

var app = builder.Build();

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

How Extension Method Works

Important syntax:

```
this IServiceCollection services
```

This tells compiler:

"Attach this method to IServiceCollection type"

So now:

```
builder.Services.AddApplicationServices();
```

looks like method belongs to `builder.Services`.

But internally compiler converts it into:

```
ServiceCollectionExtensions.AddApplicationServices(builder.Services);
```

REAL INTERNAL CONVERSION

What You Write

```
builder.Services.AddApplicationServices();
```

Compiler Converts To

```
ServiceCollectionExtensions.AddApplicationServices(builder.Services);
```

This is the most important concept.

Why Use Extension Method?

Cleaner Syntax

Without Extension Method

```
ServiceCollectionExtensions.AddApplicationServices(builder.Services);  
ServiceCollectionExtensions.AddDatabaseServices(builder.Services);  
ServiceCollectionExtensions.AddJwtServices(builder.Services);
```

With Extension Method

```
builder.Services.AddApplicationServices();  
builder.Services.AddDatabaseServices();  
builder.Services.AddJwtServices();
```

Cleaner and easier to read.

Method Chaining

Because method returns:

```
IServiceCollection
```

we can do:

```
builder.Services  
    .AddApplicationServices()  
    .AddDatabaseServices()  
    .AddJwtServices();
```

This is called:

Fluent API Style

Used heavily in ASP.NET Core.

Easy Real-World Example

Extension Method Example

```
public static class StringExtensions
{
    public static void Print(this string text)
    {
        Console.WriteLine(text);
    }
}
```

Usage

```
"Hello".Print();
```

Internally Compiler Converts To

```
StringExtensions.Print("Hello");
```

Key Rule

If method parameter has:

```
this SomeType value
```

then it is an Extension Method.

Difference Summary

Feature	Normal Static Method	Extension Method
Call Style	ClassName.Method()	object.Method()

Feature	Normal Static Method	Extension Method
Needs <code>this</code> keyword	✗ No	✓ Yes
Cleaner Syntax	✗	✓
Supports Method Chaining	✗	✓
Enterprise Usage	Less	Very Common

Final Understanding

Normal Static Method

```
ServiceCollectionExtensions.AddApplicationServices(builder.Services);
```

Method belongs to class.

Extension Method

```
builder.Services.AddApplicationServices();
```

Looks like object method.

But internally:

```
ServiceCollectionExtensions.AddApplicationServices(builder.Services);
```

same thing happens.

Compiler magic only.

Senior Developer Note

Most ASP.NET Core built-in methods are Extension Methods:

```
builder.Services.AddControllers();  
builder.Services.AddSwaggerGen();  
builder.Services.AddDbContext<AppDbContext>();
```

Microsoft uses this pattern everywhere for clean architecture and fluent API design.